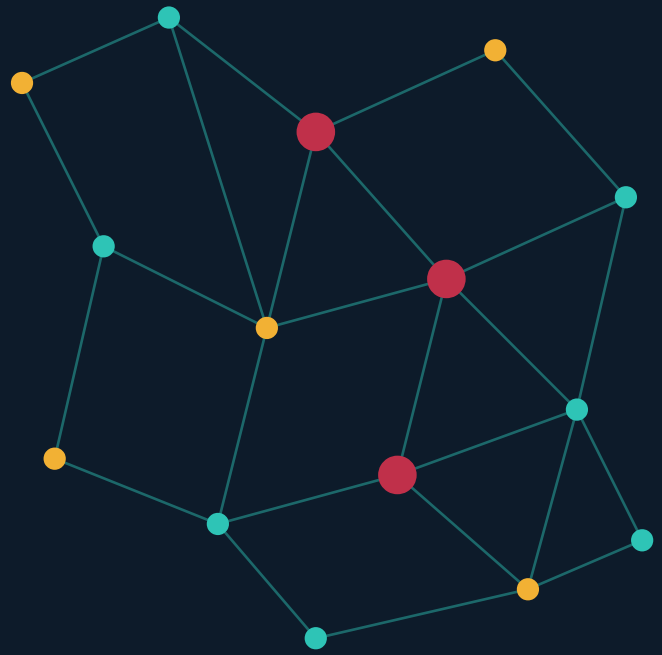
ΕΒΔΟΜΑΔΑ 5

Ερωτήματα μέσα σε ερωτήματα



CASE WHEN & συναρτήσεις · υποερωτήματα · EXISTS · CTE · πίνακες ## · Views · star & snowflake

Δρ. Αθανάσιος Κρανιάς
Βάσεις Δεδομένων & Επιχειρηματική Αναλυτική · Τμήμα Χρηματοοικονομικής και Τραπεζικής Διοικητικής · Πανεπιστήμιο Πειραιώς

Πού είμαστε

Η τελευταία εβδομάδα της ενότητας SQL — από την επόμενη, οικονομετρία

Εβδομάδα 4 σε 30″

- INNER: μόνο ζευγάρια · LEFT: όλα από αριστερά
- LEFT JOIN + IS NULL = «τι λείπει»
- Σχέσεις 1:1, 1:N, N:M
- COUNT πριν και μετά το JOIN



Σήμερα θα μάθεις να:

- φτιάχνεις κατηγορίες με CASE WHEN και χειρίζεσαι NULL, ημερομηνίες, κείμενο
- χρησιμοποιείς το αποτέλεσμα ενός ερωτήματος μέσα σε άλλο
- απαντάς «υπάρχει;» με EXISTS
- σπας ένα δύσκολο ερώτημα σε βήματα: CTE & πίνακες ##
- αποθηκεύεις ερωτήματα ως Views για αναφορές

Από σήμερα: η βάση UniStyleDB

Το eshop ρούχων που ήδη ξέρετε από το Excel — τώρα ως βάση στον SQL Server

Πίνακας	Γραμμές	Τι περιέχει
Customers	400	Πελάτες (η πόλη όπως την έγραψε ο πελάτης)
Cities	13	Πόλεις → περιφέρειες
Products	44	Κατάλογος: κατηγορία, μάρκα, κόστος, τιμή
Orders	2.715	Παραγγελίες: πελάτης, ημερομηνία, κανάλι, κατάσταση
OrderItems	4.977	Γραμμές παραγγελιών (N:M Orders–Products)
DailyMarketing	731	Διαφήμιση & newsletter ανά ημέρα

Τα δεδομένα είναι φανταστικά · ίδια με το UniStyle_eShop_Data.xlsx (v2)

Πώς τη φορτώνω

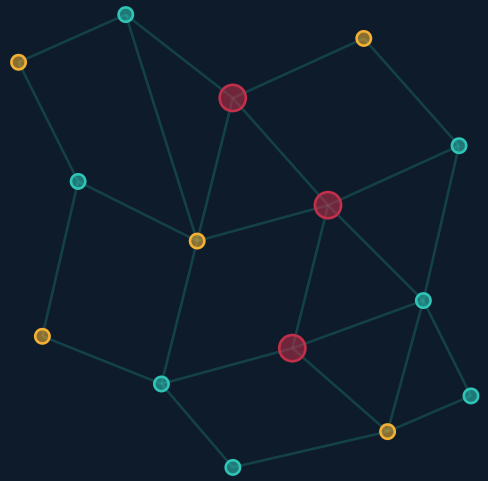
Laptop: UniStyleDB.sql → F5. Φτιάχνει τη βάση με όλα τα κλειδιά.

Εργαστήριο: UniStyleDB_temp_tables.sql → F5, και αφήνετε το παράθυρο ανοιχτό. Οι πίνακες έχουν ## μπροστά: SELECT * FROM ##Orders;

01

Η εργαλειοθήκη της SQL

CASE WHEN · NULL · ημερομηνίες · κείμενο · CAST — μικρά εργαλεία που θα χρησιμοποιούμε συνέχεια



CASE WHEN: το «IF» της SQL

Φτιάχνουμε κατηγορίες από αριθμούς — εδώ: ζώνες τιμής για τα προϊόντα

```sql
SELECT ProductName, ListPrice,
       CASE WHEN ListPrice < 30 THEN N'Οικονομικό'
            WHEN ListPrice < 80 THEN N'Μεσαίο'
            ELSE N'Ακριβό'
       END AS PriceBand
FROM Products;
```

ΚΑΙ ΜΕ GROUP BY

```sql
SELECT CASE WHEN ListPrice < 30 THEN N'Οικονομικό'
            WHEN ListPrice < 80 THEN N'Μεσαίο'
            ELSE N'Ακριβό' END AS PriceBand,
       COUNT(*) AS Products, AVG(ListPrice) AS AvgPrice
FROM Products
GROUP BY CASE WHEN ListPrice < 30 THEN N'Οικονομικό'
              WHEN ListPrice < 80 THEN N'Μεσαίο'
              ELSE N'Ακριβό' END;
```

PriceBand	Products	AvgPrice
Οικονομικό	9	21.21
Μεσαίο	25	55.55
Ακριβό	10	124.29

Με τη σειρά: το πρώτο WHEN που ισχύει κερδίζει — γι' αυτό το δεύτερο γράφει μόνο < 80

Χωρίς ELSE: ό,τι δεν ταιριάζει γίνεται NULL

Στο GROUP BY γράφουμε ξανά όλο το CASE — όχι το ψευδώνυμο

Ίδιο με: IF του Excel · case_when() της R (εβδ. 9)

CASE μέσα σε SUM: ένα «pivot» με SQL

Παραδοτέες παραγγελίες ανά έτος — App και Web σε ξεχωριστές στήλες

```sql
SELECT YEAR(OrderDate) AS SalesYear,
       SUM(CASE WHEN Channel = 'App' THEN 1 ELSE 0 END) AS AppOrders,
       SUM(CASE WHEN Channel = 'Web' THEN 1 ELSE 0 END) AS WebOrders,
       COUNT(*) AS AllOrders
FROM Orders
WHERE Status = 'Delivered'
GROUP BY YEAR(OrderDate)
ORDER BY SalesYear;
```

SalesYear	AppOrders	WebOrders	AllOrders	Μερίδιο App
2024	334	904	1238	27,0%
2025	543	844	1387	39,1%



Όπως ένα Pivot Table: γραμμές = έτη, στήλες = κανάλια. Κάθε CASE «μετράει» μόνο τις γραμμές που ταιριάζουν (1), τις άλλες τις μετράει 0.

NULL: ISNULL & COALESCE

Αντικαθιστούμε το «κενό» με μια τιμή — για καθαρές αναφορές και σωστούς υπολογισμούς

```sql
SELECT COALESCE(i.ReturnReason, N'Δεν επιστράφηκε') AS Reason,
       COUNT(*) AS Lines
FROM OrderItems i
JOIN Orders o ON o.OrderID = i.OrderID
WHERE o.Status = 'Delivered'
GROUP BY COALESCE(i.ReturnReason, N'Δεν επιστράφηκε')
ORDER BY Lines DESC;
```

Reason	Lines
Δεν επιστράφηκε	4253
Wrong size	318
Not as described	109
Changed mind	97
Defective	29

Συνάρτηση	Τι κάνει
`ISNULL(x, τιμή)`	Αν το x είναι NULL, βάζει την τιμή (SQL Server)
`COALESCE(x, y, z)`	Η πρώτη τιμή που δεν είναι NULL (κάθε βάση)
`COUNT(*) vs COUNT(στήλη)`	Όλες οι γραμμές vs όσες δεν είναι NULL: 400 vs 378 στο BirthDate

Συναρτήσεις ημερομηνίας

Από μία στήλη OrderDate βγαίνουν έτος, μήνας, τρίμηνο, ηλικία…

Συνάρτηση	Παράδειγμα	Αποτέλεσμα
YEAR / MONTH / DAY	MONTH('2025-11-28')	11
DATEPART(QUARTER, d)	DATEPART(QUARTER, '2025-11-28')	4
DATENAME(WEEKDAY, d)	DATENAME(WEEKDAY, '2025-11-28')	Friday
DATEDIFF(DAY, a, b)	DATEDIFF(DAY, '2025-11-01', '2025-11-28')	27
DATEADD(DAY, n, d)	DATEADD(DAY, 30, '2025-11-28')	2025-12-28
EOMONTH(d)	EOMONTH('2025-11-28')	2025-11-30

```sql
SELECT DATEPART(QUARTER, OrderDate) AS Q,
       COUNT(*) AS Orders
FROM Orders
WHERE Status = 'Delivered'
  AND YEAR(OrderDate) = 2025
GROUP BY DATEPART(QUARTER, OrderDate)
ORDER BY Q;
-- Q1 307 · Q2 326 · Q3 272 · Q4 482
```

Ηλικία πελάτη: DATEDIFF(YEAR, BirthDate, '2025-12-31') AS Age. Προσοχή: το DATEDIFF μετρά «πόσες αλλαγές έτους» μεσολαβούν (2025 − έτος γέννησης), όχι την ακριβή ηλικία — για αναλύσεις ανά ηλικιακή ομάδα αρκεί.

Συναρτήσεις κειμένου & «βρώμικα» δεδομένα

Η πόλη του πελάτη είναι όπως την πληκτρολόγησε: « Athens», «ATHENS», «Athina»…

Συνάρτηση	Παράδειγμα → αποτέλεσμα
UPPER / LOWER	UPPER('Chania') → CHANIA
TRIM	TRIM(' Athens ') → Athens
LEN	LEN('Rhodes') → 6
LEFT(x, n)	LEFT('Thessaloniki', 4) → Thes
+ ή CONCAT	FirstName + ' ' + LastName

Ο SQL Server «συγχωρεί»: δεν κάνει διάκριση κεφαλαίων ('athens' = 'Athens') και αγνοεί τα κενά στο τέλος. Δεν συγχωρεί τα κενά στην αρχή (« Athens») ούτε τα λάθη («Athina»).

```sql
-- 1. Ποιοι πελάτες δεν «ταιριάζουν» με τον Cities;
SELECT c.CustomerID, c.City
FROM Customers c
LEFT JOIN Cities ct ON ct.City = c.City
WHERE ct.City IS NULL;              -- 4 πελάτες

-- 2. Καθάρισμα
UPDATE Customers SET City = TRIM(City);
UPDATE Customers SET City = 'Athens'
WHERE City = 'Athina';
-- ξανά το 1 → 0 πελάτες
```

Το UPDATE αλλάζει μόνιμα τα δεδομένα: πρώτα SELECT για να δείτε τι θα αλλάξει.

CAST & η παγίδα της ακέραιας διαίρεσης

Όταν δουλεύεις με ακέραιους, ο SQL Server δίνει ακέραιο αποτέλεσμα — χωρίς προειδοποίηση

```
SELECT 5 / 2;                                   -- 2   (!)
SELECT 5 / 2.0;                                 -- 2.5

SELECT AVG(Quantity) FROM OrderItems;           -- 1   (!)
SELECT AVG(CAST(Quantity AS DECIMAL(6,2)))
FROM OrderItems;                                -- 1.12

SELECT AVG(Returned) FROM OrderItems;           -- σφάλμα: BIT
SELECT AVG(CAST(Returned AS FLOAT))
FROM OrderItems;                                -- 0.115
```

Κανόνες

- INT / INT = INT: το δεκαδικό «κόβεται»
- CAST(x AS DECIMAL / FLOAT) πριν τη διαίρεση ή τον μέσο όρο
- Γρήγορο κόλπο: x * 1.0
- AVG μιας στήλης 0/1 = ποσοστό: 0,115 → 11,5% των γραμμών επιστράφηκαν

Μια ερώτηση, δύο βήματα

«Ποια προϊόντα είναι ακριβότερα από τη μέση τιμή του καταλόγου;»

```sql
-- Βήμα 1: ποια είναι η μέση τιμή;
SELECT AVG(ListPrice)
FROM Products;
-- 64.149090
```

```sql
-- Βήμα 2: αντιγράφω τον αριθμό
SELECT ProductName, ListPrice
FROM Products
WHERE ListPrice > 64.15;
```

Το πρόβλημα: ο αριθμός είναι «καρφωμένος». Αν αλλάξει μια τιμή ή μπει νέο προϊόν, ο μέσος όρος αλλάζει — και το ερώτημα δίνει λάθος αποτέλεσμα χωρίς να το καταλάβει κανείς.

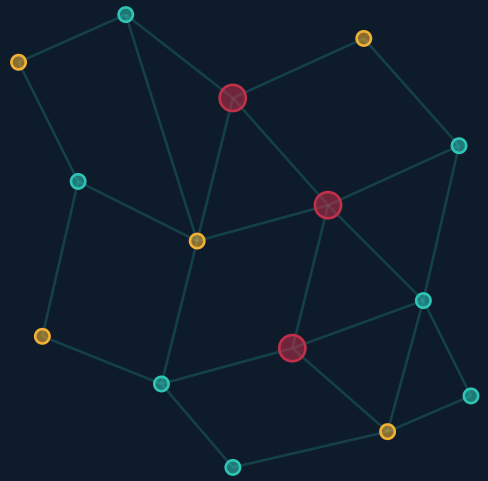
✓ ΜΕ ΥΠΟΕΡΩΤΗΜΑ: ΕΝΑ ΒΗΜΑ, ΠΑΝΤΑ ΣΩΣΤΟ

```sql
SELECT ProductName, ListPrice
FROM Products
WHERE ListPrice > (SELECT AVG(ListPrice) FROM Products);
```

02

Υποερωτήματα

Subqueries: ένα SELECT μέσα σε παρενθέσεις



Υποερώτημα που επιστρέφει μία τιμή

Scalar subquery: εκτελείται πρώτο, και το αποτέλεσμά του μπαίνει στη θέση του

```
SELECT ProductName, Category, ListPrice
FROM Products
WHERE ListPrice >
      (SELECT AVG(ListPrice)
       FROM Products)
ORDER BY ListPrice DESC;
```

Πρώτα τρέχει το εσωτερικό ερώτημα → 64,15

Μετά το εξωτερικό: WHERE ListPrice > 64,15

Πρέπει να επιστρέφει ακριβώς μία γραμμή, μία στήλη

ProductName	Category	ListPrice
Bomber Jacket	Jackets	166.99
Trench Coat	Jackets	162.99
Denim Jacket	Jackets	153.99
Wool Coat	Jackets	132.99
Puffer Jacket	Jackets	124.99
Ankle Boots	Shoes	122.99
…	…	…

18 από τα 44 προϊόντα

Πού μπαίνει ένα υποερώτημα;

Σε τρία σημεία — με διαφορετικό ρόλο το καθένα

WHERE — Ως φίλτρο

```
WHERE ListPrice >
  (SELECT AVG(…) …)
```

Σύγκριση με τιμή ή λίστα τιμών

SELECT — Ως νέα στήλη

```
SELECT ProductName,
  ListPrice,
  (SELECT AVG(…) …)
  AS CatalogAvg
```

Ίδια τιμή σε κάθε γραμμή

FROM — Ως «προσωρινός» πίνακας

```
SELECT …
FROM (SELECT …) AS t
```

Derived table — πρέπει να έχει ψευδώνυμο

Υποερώτημα στο FROM

«Πρώτα υπολόγισε, μετά συνόψισε» · μέση αξία παραγγελίας ανά τρόπο πληρωμής, 2025

```sql
SELECT t.PaymentMethod,
       AVG(t.OrderValue) AS AvgOrder,
       COUNT(*)          AS Orders
FROM (SELECT o.OrderID, o.PaymentMethod,
             SUM(i.Quantity * i.UnitPrice
                 * (1 - i.DiscountPct)) AS OrderValue
      FROM Orders o
      JOIN OrderItems i ON i.OrderID = o.OrderID
      WHERE o.Status = 'Delivered'
        AND YEAR(o.OrderDate) = 2025
      GROUP BY o.OrderID, o.PaymentMethod) AS t
GROUP BY t.PaymentMethod
ORDER BY AvgOrder DESC;
```

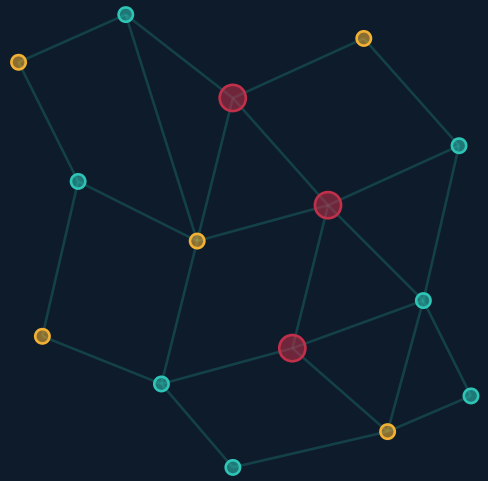
PaymentMethod	AvgOrder €	Orders
Cash on Delivery	138.57	340
Card	121.85	740
PayPal	119.78	189
Apple Pay	109.26	118

Γιατί δύο βήματα; Η «αξία παραγγελίας» δεν υπάρχει σε καμία στήλη: πρώτα αθροίζουμε τις γραμμές κάθε παραγγελίας (εσωτερικό), μετά βγάζουμε μέσο όρο (εξωτερικό). Ένα AVG κατευθείαν στις γραμμές θα έδινε τη μέση αξία γραμμής — άλλη ερώτηση.

03

IN, EXISTS, ANY, ALL

Υποερωτήματα που επιστρέφουν πολλές τιμές



IN με υποερώτημα

Το υποερώτημα επιστρέφει μια λίστα — το IN ελέγχει αν η τιμή είναι μέσα

```sql
-- Ποια προϊόντα έχουν πουληθεί με έκπτωση 50%;
SELECT ProductName, Category
FROM Products
WHERE ProductID IN (
    SELECT ProductID
    FROM OrderItems
    WHERE DiscountPct = 0.50
);
```

ProductName	Category
Basic Tee	T-shirts
Graphic Tee	T-shirts
Midi Dress	Dresses
Maxi Dress	Dresses
…	…
21 προϊόντα	

Υποερώτημα → λίστα με ProductID · εξωτερικό → WHERE ProductID IN (101, 102, …)

Το IN «σβήνει» τις διπλοεγγραφές: κάθε προϊόν εμφανίζεται μία φορά, όσες φορές κι αν πουλήθηκε (με JOIN θα εμφανιζόταν πολλές)

NOT IN — προσοχή αν η λίστα περιέχει NULL: δεν επιστρέφει τίποτα!

EXISTS: «υπάρχει τουλάχιστον μία γραμμή;»

Συσχετισμένο υποερώτημα (correlated): εκτελείται για κάθε γραμμή του εξωτερικού

```sql
-- Πελάτες με τουλάχιστον μία επιστροφή
SELECT c.CustomerID, c.LastName
FROM Customers AS c
WHERE EXISTS (
    SELECT 1
    FROM Orders o JOIN OrderItems i
         ON i.OrderID = o.OrderID
    WHERE o.CustomerID = c.CustomerID
      AND i.Returned = 1
);
-- 212 πελάτες
```

```sql
-- Πελάτες χωρίς καμία παραγγελία
SELECT c.CustomerID, c.LastName
FROM Customers AS c
WHERE NOT EXISTS (
    SELECT 1 FROM Orders AS o
    WHERE o.CustomerID = c.CustomerID
);
-- 71 πελάτες
```

`c.CustomerID` μέσα στο υποερώτημα = η σύνδεση με την τρέχουσα γραμμή. **`SELECT 1:`** δεν μας νοιάζει τι επιστρέφει — μόνο αν επιστρέφει κάτι.

ANY & ALL: σύγκριση με μια ομάδα τιμών

Πώς συγκρίνονται οι τιμές των άλλων προϊόντων με τα παπούτσια;

> ANY: ΑΚΡΙΒΟΤΕΡΑ ΑΠΟ ΤΟΥΛΑΧΙΣΤΟΝ ΕΝΑ ΖΕΥΓΑΡΙ

```
SELECT ProductName, Category, ListPrice
FROM Products
WHERE Category <> 'Shoes'
  AND ListPrice > ANY (
      SELECT ListPrice
      FROM Products
      WHERE Category = 'Shoes')
ORDER BY ListPrice DESC;
```

Παπούτσια	ListPrice
Ankle Boots	122.99
Sandals	115.99
Loafers	97.99
Leather Sneakers	79.99
Running Shoes	65.99

Τελεστής	Ισοδύναμο	Γραμμές
> ANY (παπούτσια)	> MIN = 65.99	13
> ALL (παπούτσια)	> MAX = 122.99	5

Τιμές σε € · ANY = SOME

Συσχετισμένο υποερώτημα

Σύγκριση με τον «δικό σου» μέσο όρο: προϊόντα ακριβότερα από τη μέση τιμή της κατηγορίας τους

```sql
SELECT p.Category, p.ProductName, p.ListPrice
FROM Products AS p
WHERE p.ListPrice > (
    SELECT AVG(p2.ListPrice)
    FROM Products AS p2
    WHERE p2.Category = p.Category  -- η κατηγορία της p
)
ORDER BY p.Category;
```

Category	ProductName	ListPrice
Accessories	Wool Scarf	39.99
Dresses	Maxi Dress	81.99
Jackets	Bomber Jacket	166.99
Jeans	Slim Jeans	81.99
Shoes	Ankle Boots	122.99
T-shirts	Graphic Tee	23.99
…	…	…
23 προϊόντα		

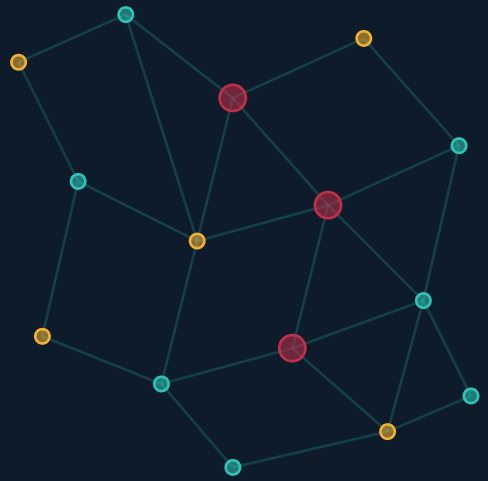


Το Graphic Tee (23,99 €) δεν είναι «ακριβό» στον κατάλογο — αλλά είναι πάνω από τον μέσο όρο των T-shirts.

04

Βήμα-βήμα: CTE & πίνακες

Πώς σπάμε ένα δύσκολο ερώτημα σε κομμάτια — και πώς κρατάμε ένα αποτέλεσμα για αργότερα



CTE (WITH): ένα ερώτημα σε βήματα

Common Table Expression: δίνουμε όνομα σε ένα υποερώτημα και το γράφουμε πρώτο

```sql
WITH OrderTotals AS (          -- βήμα 1: αξία κάθε παραγγελίας
  SELECT o.OrderID, o.PaymentMethod,
         SUM(i.Quantity * i.UnitPrice
             * (1 - i.DiscountPct)) AS OrderValue
  FROM Orders o
  JOIN OrderItems i ON i.OrderID = o.OrderID
  WHERE o.Status = 'Delivered' AND YEAR(o.OrderDate) = 2025
  GROUP BY o.OrderID, o.PaymentMethod
)
SELECT PaymentMethod,            -- βήμα 2: μέσος όρος
       AVG(OrderValue) AS AvgOrder, COUNT(*) AS Orders
FROM OrderTotals
GROUP BY PaymentMethod
ORDER BY AvgOrder DESC;
```

Ίδιο αποτέλεσμα με τη διαφάνεια 15

- Διαβάζεται από πάνω προς τα κάτω — όπως σκεφτόμαστε
- Το όνομα (OrderTotals) εξηγεί τι κάνει το βήμα
- Πολλά βήματα: WITH a AS (…), b AS (…) SELECT …
- Δουλεύει παντού — και στο εργαστήριο

Πίνακας ##: κρατάω ένα αποτέλεσμα για αργότερα

SELECT … INTO ##Όνομα: φτιάχνει προσωρινό πίνακα με το αποτέλεσμα — μετά τον ρωτάω όσες φορές θέλω

```sql
DROP TABLE IF EXISTS ##CustomerSales;

SELECT c.CustomerID, c.FirstName, c.LastName,
       COUNT(DISTINCT o.OrderID) AS NumOrders,
       SUM(i.Quantity * i.UnitPrice
           * (1 - i.DiscountPct)) AS TotalSpend
INTO   ##CustomerSales
FROM   Customers c
JOIN   Orders o     ON o.CustomerID = c.CustomerID
JOIN   OrderItems i ON i.OrderID    = o.OrderID
WHERE  o.Status = 'Delivered'
GROUP BY c.CustomerID, c.FirstName, c.LastName;
```

ΜΕΤΑ: ΟΣΑ ΕΡΩΤΗΜΑΤΑ ΘΕΛΩ

```sql
SELECT TOP 5 * FROM ##CustomerSales
ORDER BY TotalSpend DESC;

SELECT AVG(TotalSpend) FROM ##CustomerSales;

SELECT COUNT(*) FROM ##CustomerSales
WHERE TotalSpend > 1000;
```

Έλεγχος: 328 πελάτες · πρώτη η Panagiota Lambrou (8.278 €) · μέσος όρος 994 € · 118 πελάτες πάνω από 1.000 €

CTE, ## ή VIEW;

Τρεις τρόποι να «κρατήσεις» ένα ερώτημα — ο καθένας για άλλη δουλειά

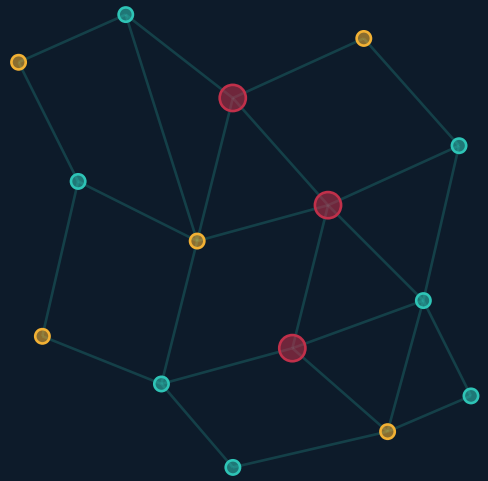
	CTE (WITH)	Πίνακας ##	VIEW
Τι κρατάει	Μόνο το ερώτημα, για μία εκτέλεση	Αντίγραφο των δεδομένων τη στιγμή που φτιάχτηκε	Τον ορισμό του ερωτήματος
Πόσο ζει	Όσο τρέχει το ερώτημα	Μέχρι να κλείσει το παράθυρο	Μόνιμα, μέσα στη βάση
Αν αλλάξουν τα δεδομένα	Πάντα φρέσκο	Δεν ενημερώνεται	Πάντα φρέσκο
Στο εργαστήριο	✓	✓	× (χωρίς δικαίωμα)
Για R, Power BI, συναδέλφους	×	×	✓

Κανόνας: CTE για να γράψω καθαρά ένα ερώτημα · ## όταν θέλω να ξαναχρησιμοποιήσω ένα «βαρύ» αποτέλεσμα σε πολλά ερωτήματα · VIEW όταν το ίδιο ερώτημα πρέπει να το χρησιμοποιούν άλλοι και άλλα εργαλεία, μόνιμα.

05

Views

Αποθηκευμένα ερωτήματα για reporting — η γέφυρα προς την R και το Power BI



Τι είναι ένα View;

Ένα αποθηκευμένο SELECT με όνομα — συμπεριφέρεται σαν πίνακας, αλλά δεν αποθηκεύει δεδομένα

```
CREATE VIEW vw_OrderTotals AS
SELECT o.OrderID, o.CustomerID, o.OrderDate,
       o.Channel, o.Status,
       SUM(i.Quantity * i.UnitPrice
           * (1 - i.DiscountPct)) AS OrderValue
FROM Orders o
JOIN OrderItems i ON i.OrderID = o.OrderID
GROUP BY o.OrderID, o.CustomerID, o.OrderDate,
         o.Channel, o.Status;
GO

-- Από εδώ και πέρα: σαν να ήταν πίνακας
SELECT AVG(OrderValue) FROM vw_OrderTotals WHERE Channel = 'App';
```



Απλότητα

Το JOIN και το SUM γράφονται μία φορά — οι χρήστες βλέπουν έναν «καθαρό» πίνακα



Συνέπεια

Όλες οι αναφορές χρησιμοποιούν τον ίδιο ορισμό της «αξίας παραγγελίας»



Ασφάλεια

Δίνουμε πρόσβαση στο view, όχι στους πίνακες με τα στοιχεία των πελατών

Εργαστήριο: χωρίς CREATE VIEW — το ίδιο SELECT τρέχει ως CTE.

Κανόνες για Views στον SQL Server

Τέσσερις λεπτομέρειες που θα συναντήσετε στο SSMS

GO πριν και μετά

Το CREATE VIEW πρέπει να είναι μόνο του στο «batch». Χωρίς GO → «CREATE VIEW must be the only statement in the batch».

```
GO
CREATE VIEW …
GO
```

Χωρίς ORDER BY

Ένα view είναι «πίνακας» — οι πίνακες δεν έχουν σειρά. Η ταξινόμηση μπαίνει στο SELECT που διαβάζει το view.

```
SELECT * FROM vw_…
ORDER BY …;
```

Αλλαγή & διαγραφή

CREATE OR ALTER ενημερώνει τον ορισμό χωρίς DROP. DROP VIEW το σβήνει — τα δεδομένα μένουν ανέγγιχτα.

```
CREATE OR ALTER VIEW …
DROP VIEW vw_…;
```

Όχι πάνω σε

Ένα view δεν μπορεί να διαβάζει προσωρινό πίνακα: θα έπαυε να δουλεύει μόλις κλείσει το παράθυρο.

```
-- ✗ σφάλμα
CREATE VIEW v AS
SELECT * FROM ##T;
```

Ένα view για reporting: vw_SalesFlat

Το φύλλο Sales_Flat του Excel, φτιαγμένο από τους πίνακες της βάσης

```sql
CREATE OR ALTER VIEW vw_SalesFlat AS
SELECT i.OrderItemID, o.OrderDate, o.Channel, o.Status,
       p.Category, p.Brand, ct.Region,
       ROUND(i.Quantity * i.UnitPrice
             * (1 - i.DiscountPct), 2) AS NetAmount
FROM OrderItems i
JOIN Orders    o  ON o.OrderID    = i.OrderID
JOIN Products  p  ON p.ProductID  = i.ProductID
JOIN Customers c  ON c.CustomerID = o.CustomerID
LEFT JOIN Cities ct ON ct.City = TRIM(c.City);
GO
```

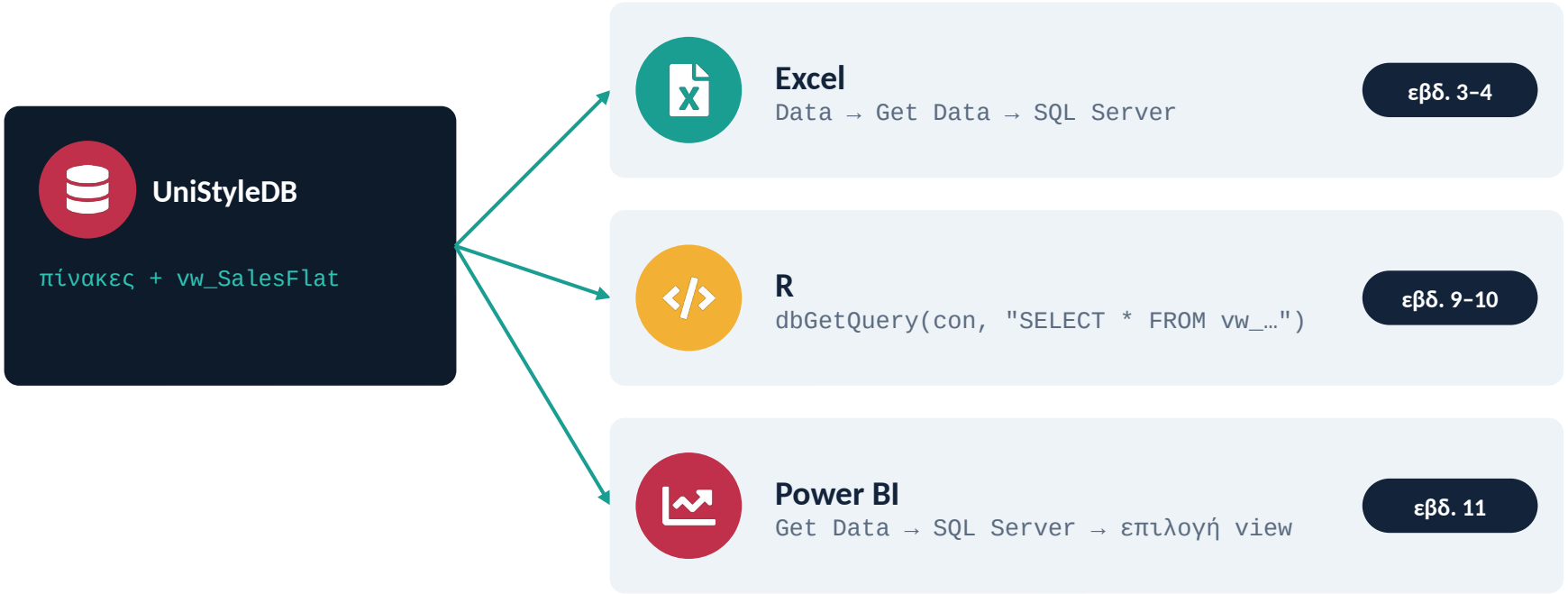
Έλεγχος

`SELECT COUNT(*)` → 4.977
SUM(NetAmount) για Status = 'Delivered' → 326.118,25 € = η «Σύνοψη» του Excel

Γιατί LEFT JOIN και TRIM; η πόλη του πελάτη είναι «βρώμικη» (« Athens», «ATHENS»). Το TRIM σβήνει τα κενά· τα κεφαλαία δεν πειράζουν. Η «Athina» μένει χωρίς περιφέρεια (NULL).

Γιατί τα Views μετράνε για το υπόλοιπο εξάμηνο

Το ίδιο view τροφοδοτεί όλα τα εργαλεία — ένας ορισμός, μία «αλήθεια»

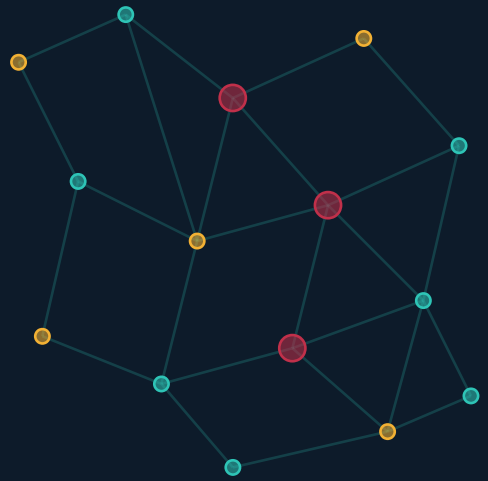


Στο εργαστήριο το ίδιο ρόλο παίζει ένα CTE που σώζουμε σε αρχείο .sql και ξανατρέχουμε.

06

Συναλλαγές & ανάλυση

OLTP & OLAP · fact & dimension tables · star & snowflake



OLTP ή OLAP;

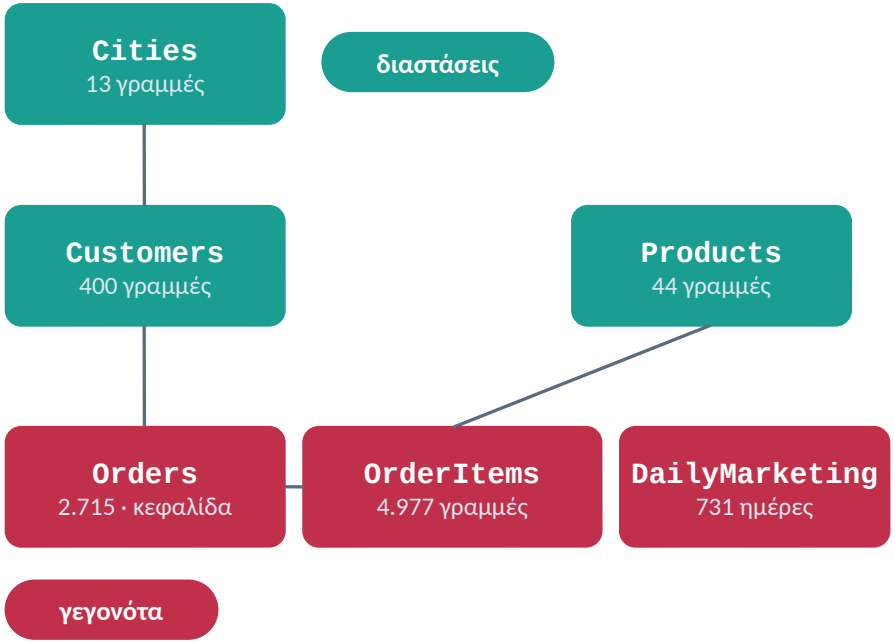
Δύο τρόποι να οργανώσεις μια βάση — ανάλογα με το ποιος τη χρησιμοποιεί και για ποιο λόγο

	OLTP · βάση συναλλαγών	OLAP · βάση ανάλυσης
Τα αρχικά	Online Transaction Processing = επεξεργασία συναλλαγών σε πραγματικό χρόνο	Online Analytical Processing = αναλυτική επεξεργασία δεδομένων
Σκοπός	Καταχώρηση των καθημερινών συναλλαγών	Ανάλυση και λήψη αποφάσεων
Παράδειγμα	Το σύστημα παραγγελιών του eshop · core banking	Αποθήκη δεδομένων (data warehouse), BI
Τυπικές εντολές	Πολλά μικρά INSERT / UPDATE	Λίγα μεγάλα SELECT με JOIN & GROUP BY
Δεδομένα	Η τρέχουσα κατάσταση	Ιστορικό ετών
Σχεδιασμός	Κανονικοποιημένος (πολλοί μικροί πίνακες)	Star ή snowflake schema
Χρήστες	Εφαρμογές, υπάλληλοι, πελάτες	Αναλυτές, διοίκηση, dashboards

Στο UniStyle: η UniStyleDB είναι η βάση συναλλαγών (OLTP) · το φύλλο Sales_Flat του Excel είναι μια μικρή «βάση ανάλυσης» (OLAP).

Πίνακες γεγονότων & πίνακες διαστάσεων

Fact tables: τι μετράμε · Dimension tables: ποιος, τι, πότε — στη δική μας βάση



Fact table

Αριθμοί που μετράμε (ποσότητα, τιμή, έκπτωση, διαφήμιση) + ξένα κλειδιά. Πολλές γραμμές — μεγαλώνει κάθε μέρα.

Dimension table

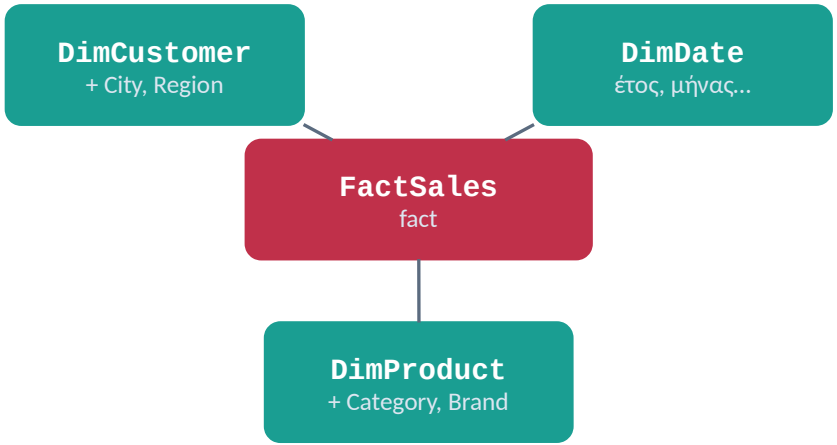
Περιγραφές: ποιος πελάτης, ποιο προϊόν, ποια πόλη. Λίγες γραμμές — αλλάζει σπάνια.

Κανόνας: τα facts τα αθροίζουμε (SUM, AVG) · με τις διαστάσεις φιλτράρουμε και ομαδοποιούμε (WHERE, GROUP BY).

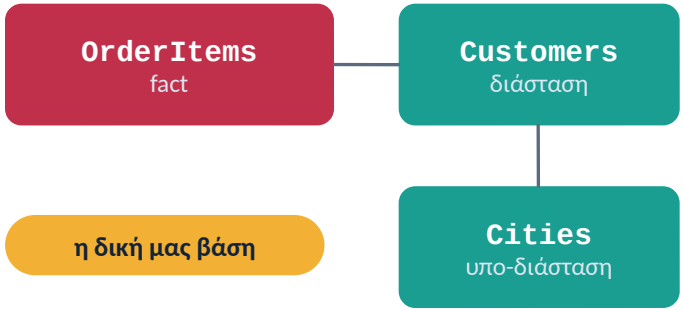
Star ή snowflake schema;

Δύο τρόποι να «δέσεις» τις διαστάσεις γύρω από τα γεγονότα

Star (αστέρι)



Snowflake (χιονονιφάδα)

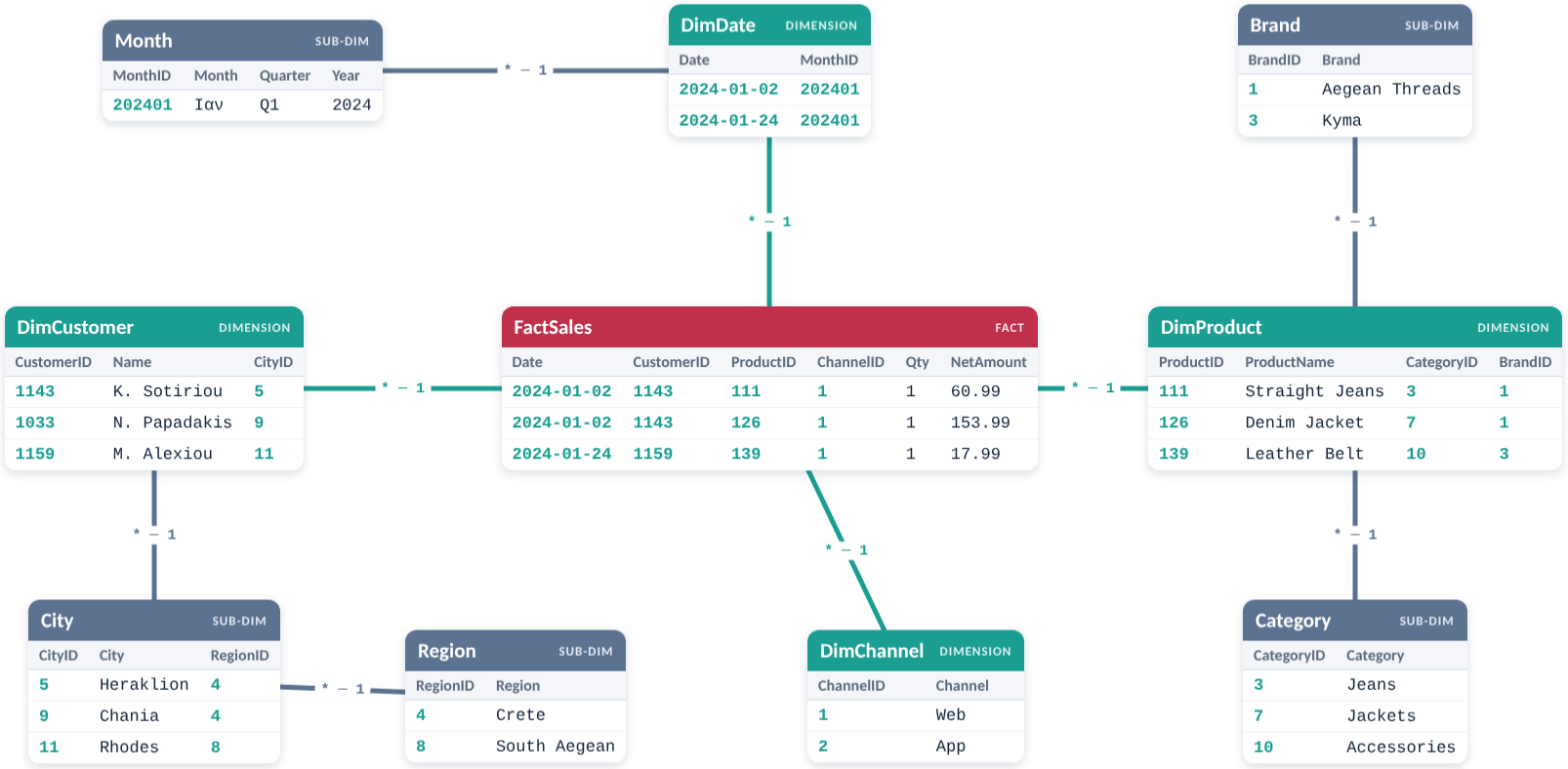


	Star	Snowflake
JOINs για ένα ερώτημα	Λίγα — ένα ανά διάσταση	Περισσότερα (αλυσίδες)
Επανάληψη κειμένου	Ναι (η περιφέρεια σε κάθε πελάτη)	Όχι — κανονικοποιημένο
Πού ταιριάζει	BI, Power BI, dashboards	Μεγάλες, «μαζεμένες» διαστάσεις

Snowflake schema · UniStyle eshop

χιονονιφάδα · κανονικοποιημένο

10 πίνακες. Οι διαστάσεις «σπάνε» σε υπο-διαστάσεις · «πωλήσεις ανά περιφέρεια» = FactSales → DimCustomer → City → Region (3 JOIN)

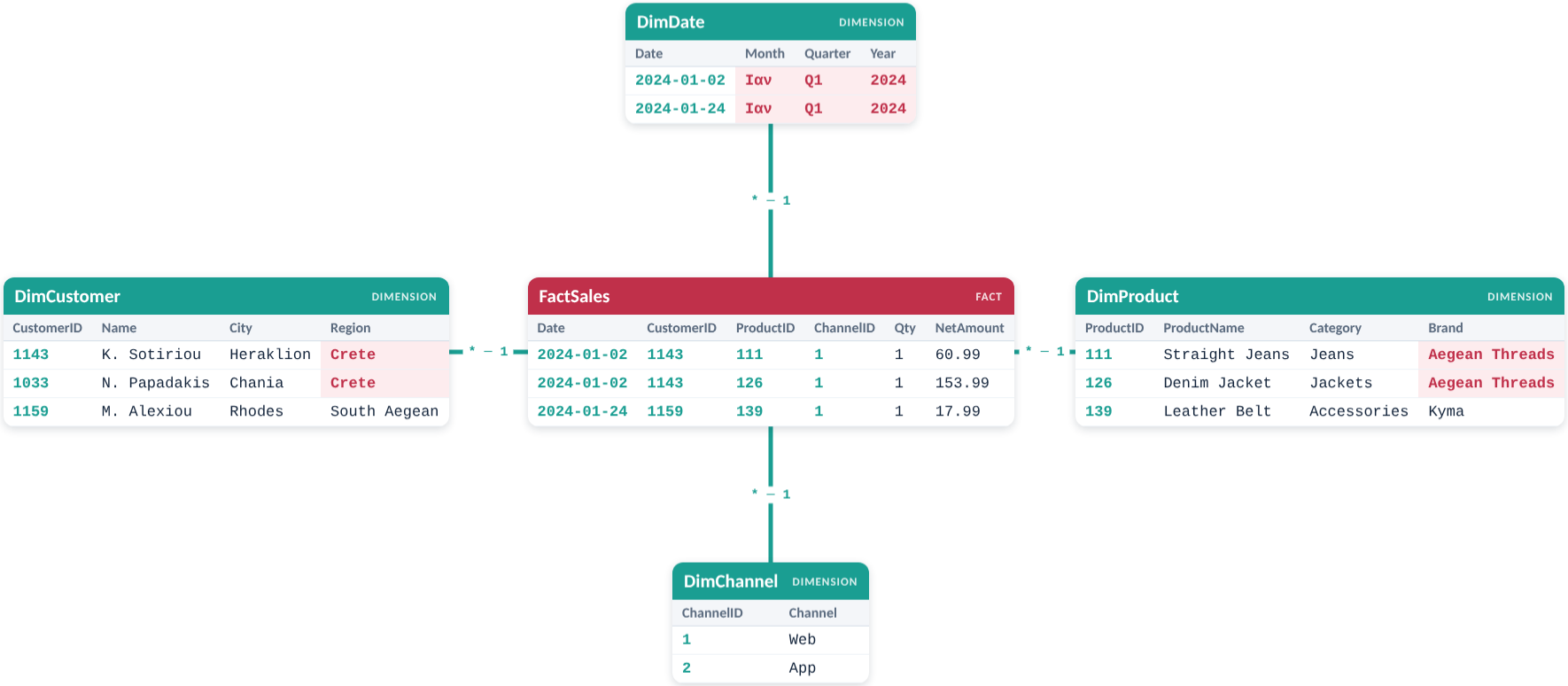


Πίνακας γεγονότων (fact): τα νούμερα · Διάσταση: ποιος · τι · πότε · Υπο-διάσταση (μόνο στο snowflake) · Τιμή που επαναλαμβάνεται

Star schema · UniStyle eshop

5 πίνακες. Κάθε διάσταση απέχει 1 JOIN από το κέντρο · «πωλήσεις ανά περιφέρεια» = FactSales → DimCustomer

αστέρι · για ανάλυση



Πίνακας γεγονότων (fact): τα νούμερα · Διάσταση: ποιος · τι · πότε · Υπο-διάσταση (μόνο στο snowflake) · Τιμή που επαναλαμβάνεται

Κανονικοποίηση σε μία διαφάνεια

Γιατί η βάση μας έχει έξι πίνακες και όχι έναν μεγάλο «Excel» όπως το Sales_Flat

Ένας μεγάλος πίνακας (μη κανονικοποιημένος)

OrderID	City	Region	ProductName	Brand
500002	Heraklion	Crete	Straight Jeans	Aegean Threads
500002	Heraklion	Crete	Crew Sweater	Nordic Pine
500002	Heraklion	Crete	Denim Jacket	Aegean Threads
…	…	…	…	…

Ενημέρωση: αλλάζει το όνομα της μάρκας Kyma → 542 αλλαγές

Εισαγωγή: νέα πόλη χωρίς πελάτες (Κέρκυρα) → δεν «χωράει»

Διαγραφή: σβήνουμε τον μοναδικό πελάτη μιας πόλης → χάνεται και η περιφέρειά της

1NF — **Μία τιμή ανά κελί**
όχι «Jeans, Shoes» σε ένα πεδίο

2NF — **Κάθε στήλη εξαρτάται από όλο το κλειδί**
το όνομα προϊόντος δεν εξαρτάται από την παραγγελία → Products

3NF — **Καμία στήλη δεν εξαρτάται από άλλη μη-κλειδί**
Region εξαρτάται από City → Cities

Κανονικοποίηση = κάθε πληροφορία γράφεται μία φορά. Το τίμημα: περισσότερα JOINs.

Views: από snowflake σε star

Δεν αλλάζουμε τους πίνακες — φτιάχνουμε τις διαστάσεις ως views

```sql
-- Διάσταση πελάτη: Customers + Cities σε ένα
CREATE OR ALTER VIEW vw_DimCustomer AS
SELECT c.CustomerID, c.Gender,
       TRIM(c.City) AS City, ct.Region
FROM   Customers c
LEFT JOIN Cities ct
       ON ct.City = TRIM(c.City);
GO
```

400 γραμμές

```sql
-- Διάσταση ημερομηνίας
CREATE OR ALTER VIEW vw_DimDate AS
SELECT MarketingDate AS DateKey,
       YEAR(MarketingDate) AS [Year],
       DATEPART(QUARTER, MarketingDate) AS [Quarter],
       MONTH(MarketingDate) AS [Month]
FROM   DailyMarketing;
GO
```

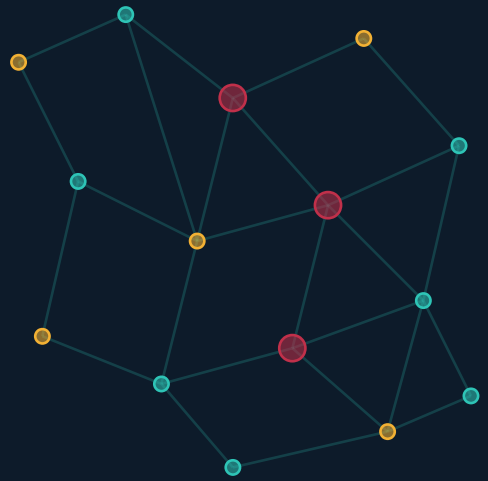
731 γραμμές — μία για κάθε ημέρα 2024–2025

Εβδ. 11 · Power BI: OrderItems + Products + vw_DimCustomer + vw_DimDate → ένα καθαρό star schema στο Model view.

07

Εργαστήριο

Εργαλειοθήκη, υποερωτήματα, CTE και πίνακες ## (εργαστήριο: ## μπροστά από κάθε πίνακα)



Άσκηση 1 · Η εργαλειοθήκη

30 λεπτά · CASE WHEN, NULL, ημερομηνίες, κείμενο, CAST

T1 Χωρίστε τα προϊόντα σε 3 ζώνες τιμής (CASE) και μετρήστε πόσα είναι σε κάθε ζώνη

T2 Παραδοτέες παραγγελίες ανά έτος, με τις στήλες AppOrders και WebOrders (SUM + CASE)

T3 Γραμμές παραγγελιών ανά λόγο επιστροφής, με «Δεν επιστράφηκε» αντί για NULL (COALESCE)

T4 Μέση ηλικία των πελατών στο τέλος του 2025, με δεκαδικά (DATEDIFF + CAST)

T5 Καθαρίστε τις πόλεις (TRIM, UPDATE) ώστε το LEFT JOIN με το Cities να μη βρίσκει κανέναν «ορφανό»

T6 Ποσοστό επιστροφών (σε γραμμές) ανά κατηγορία, από το μεγαλύτερο (CAST του Returned)

Έλεγχος: T1 → 9 / 25 / 10 · T2 → 2025: 543 App, 844 Web · T3 → 4.253 «Δεν επιστράφηκε» · T4 → 45,16 (378 πελάτες) · T5 → 4 → 0 · T6 → πρώτα τα Jeans (17,4%)

Άσκηση 2 · Υποερωτήματα

40 λεπτά · ένα ερώτημα για κάθε ερώτηση

Q1 Προϊόντα με τιμή κόστους (CostPrice) πάνω από τη μέση τιμή κόστους όλων των προϊόντων

Q2 Πόσες παραγγελίες έκαναν οι πελάτες με Newsletter = 'Yes'; (IN + υποερώτημα)

Q3 Ποιο προϊόν δεν επιστράφηκε ποτέ; (NOT EXISTS)

Q4 Για κάθε κατηγορία: πλήθος προϊόντων και, σε νέα στήλη, το συνολικό πλήθος του καταλόγου (υποερώτημα στο SELECT)

Q5 Η πιο πρόσφατη παραγγελία κάθε πελάτη (συσχετισμένο υποερώτημα με MAX)

Q6 Ο τρόπος πληρωμής με τη μεγαλύτερη μέση αξία παραγγελίας το 2025 (υποερώτημα στο FROM + TOP 1)

Έλεγχος: Q1 → 18 προϊόντα · Q2 → 1.146 παραγγελίες · Q3 → Wool Scarf · Q4 → 44 · Q5 → 333 γραμμές (329 πελάτες · 4 ισοπαλίες) · Q6 → Cash on Delivery (138,57 €)

Άσκηση 3 · CTE, πίνακες ## και (στο laptop) views

35 λεπτά · το «reporting layer» μας

α) CTE — Ξαναγράψτε το Q6 της άσκησης 2 με WITH. Βγάζει το ίδιο αποτέλεσμα;

β) ## — Φτιάξτε τον ##CustomerSales της διαφάνειας 23: ένας πελάτης ανά γραμμή, με NumOrders και TotalSpend

γ) ## — Από τον ##CustomerSales: οι 5 καλύτεροι πελάτες · η μέση αξία ανά πελάτη · πόσοι ξόδεψαν πάνω από 1.000 €

δ) VIEW — Laptop: το ίδιο ως CREATE VIEW vw_CustomerSales. Εργαστήριο: σκεφτείτε — τι θα γινόταν με τον ## αν αύριο έμπαινε νέα παραγγελία; Και με το view;

Έλεγχος: 328 πελάτες · πρώτη η Lambrou (8.278,28 €) · μέσος όρος 994,27 € · 118 πελάτες πάνω από 1.000 €

Ορόσημο: το project αρχίζει να παίρνει μορφή

Η ενότητα SQL σας έδωσε το «Βήμα 2» του end-to-end pipeline

1	2	3	4
Ερώτηση	**SQL**	**R & Οικονομετρία**	**Dashboard**
Διαλέξτε ένα επιχειρηματικό ερώτημα για το UniStyle	Γράψτε τα ερωτήματα (CTE / views) που το απαντούν ✓	εβδ. 6–10	εβδ. 11–13

Παραδείγματα ερωτημάτων για το project

- Ποιοι είναι οι πιο «πολύτιμοι» πελάτες — και τι τους χαρακτηρίζει (κανάλι, ηλικία, περιοχή);
- Τι εξηγεί τις επιστροφές; Μειώθηκαν μετά τον νέο οδηγό μεγεθών (1/3/2025);
- Αξίζει η διαφήμιση και το newsletter; (θα το απαντήσουμε με παλινδρόμηση από την εβδ. 6)

Ανάθεση DataCamp · Εβδομάδα 5

Ολοκλήρωση πριν το επόμενο μάθημα · στο DataCamp Classroom του μαθήματος

Data Manipulation in SQL
Κεφ. 2 Short and Simple Subqueries · Κεφ. 3 Correlated Subqueries & CTEs

~1,5 ώρα

Database Design
Κεφ. 3 Database Views

~1 ώρα

Προετοιμασία εβδ. 6 · Introduction to R
Κεφ. 1 Intro to basics — εγκαταστήστε R & RStudio

~30′

Θυμηθείτε: το DataCamp γράφει PostgreSQL — το LIMIT γίνεται TOP στον SQL Server.

Σύνοψη ενότητας SQL & επόμενη εβδομάδα

Τι κρατάμε από σήμερα

- CASE WHEN · ISNULL/COALESCE · DATEPART, DATEDIFF · TRIM · CAST
- Υποερώτημα = ερώτημα σε παρενθέσεις: στο WHERE, SELECT ή FROM
- IN για λίστες · EXISTS για «υπάρχει;» · ANY ≈ MIN, ALL ≈ MAX
- CTE: ένα ερώτημα σε βήματα · ##: αντίγραφο για αργότερα
- View = αποθηκευμένο SELECT · GO, χωρίς ORDER BY
- OLTP vs OLAP · facts & διαστάσεις · star vs snowflake

Εβδομάδα 6

- Νέα ενότητα: Οικονομετρία
- Απλή & πολλαπλή γραμμική παλινδρόμηση
- Το πρώτο μας μοντέλο: πόσες πωλήσεις φέρνει η διαφήμιση;
- Φέρτε εγκατεστημένα R + RStudio!